WEB APPLICATION PENETRATION TEST — VULNERABILITY REPORT

Session Fixation — Session ID Not Regenerated on Login

Finding NVP-012 · **Medium**

Product: PHPGurukul News Portal Project in PHP and MySQL — V4.1

Assessment target: http://localhost/newsportal/newsportal/

Technology: Apache 2.4.58 · PHP 8.2.12 · MariaDB 10.4.32 (DB user: root)

Assessment date: 2026-10-31

Classification: CONFIDENTIAL — authorised lab assessment (review draft)

Document Control

Finding ID	NVP-012
Vulnerability	Session Fixation — Session ID Not Regenerated on Login
Severity	**Medium**
CVSS 4.0	5.4 `CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:A/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N`
CWE	CWE-384 (Session Fixation)
OWASP	A07:2021 Identification & Authentication Failures
CVE verification	No matching publicly assigned CVE was identified for this specific finding and product/version during the research performed.
Status	CONFIRMED (manually validated & re-verified)

1. Executive Summary

The application does not regenerate the session identifier upon authentication, so a session id fixed before login remains valid after login — enabling session fixation.

2. Affected Target

Affected URL	/admin/index.php
Affected parameter	PHPSESSID
Affected role	Victim admin/sub-admin
Authentication	n/a

3. Technical Description

admin/index.php sets $_SESSION values on success but never calls session_regenerate_id(). The pre-authentication PHPSESSID is identical to the post-authentication one, so an attacker who fixes a victim's session id (e.g. via a crafted link/XSS) can ride the authenticated session.

4. Root Cause Analysis

```
admin/index.php:18-19 — $_SESSION['login']=$_POST['username'];
$_SESSION['utype']=$num['userType']; // no session_regenerate_id(true)
```

5. Preconditions

Ability to set/know the victim's session id before they log in.

6. Proof of Concept — Step-by-Step Reproduction

1. Capture the PHPSESSID issued before login.
2. Log in with valid credentials using that same cookie.
3. Observe the PHPSESSID is unchanged after authentication (pre==post), confirming fixation.

7. HTTP Request Evidence

```
(1) GET /admin/index.php -> Set-Cookie PHPSESSID=u1cmvakk...  (2) POST login with same cookie
```

8. HTTP Response Evidence

```
After login the session cookie remains PHPSESSID=u1cmvakk... (identical) — no regeneration.
```

9. Proof of Concept — Visual Evidence

> **Evidence note:** Verified live: pre-login and post-login PHPSESSID were byte-identical.

10. Actual Security Impact

An attacker who fixes a victim's session id obtains an authenticated session once the victim logs in.

Exploitation complexity	Medium — requires fixing the victim's session id and victim login.
Realistic attack scenario	Attacker plants a known session id (link/XSS), victim authenticates, attacker reuses the fixed, now-authenticated session.

11. Remediation Recommendations

Call session_regenerate_id(true) immediately after successful authentication (and on privilege change); set secure cookie attributes.

Secure code example

```
if (/* auth ok */) { session_regenerate_id(true); $_SESSION['login']=...; }
```

12. Retesting Procedure

Compare pre/post-login session ids; a fixed system issues a new id after authentication.

13. References

- CWE-384
- OWASP WSTG-SESS-03
- OWASP A07:2021

14. Conclusion

Confirmed session fixation. Verified live (identical pre/post-login session id).